

Documentação Programming Assignment 1 – Recuperação de Informação

Francisco Teixeira Rocha Aragão - 2021031726

franciscoaragao@protonmail.com

Departamento de Ciência da Computação - UFMG

Belo Horizonte, Minas Gerais, Brasil

Abstract

Máquinas de busca modernas realizam diversas tarefas para conseguir alcançar um grande número de páginas web. Desse modo, o Crawler é um módulo fundamental dessa tarefa, sendo este o objetivo desse trabalho, que realiza a implementação de um sistema que consegue crawl informações e salvar o conteúdo de páginas web, a partir de URLs inseridas pelo usuário.

CCS Concepts

• Information systems → Information retrieval.

Keywords

crawler, recuperação, informação

ACM Reference Format:

Francisco Teixeira Rocha Aragão - 2021031726. 2018. Documentação Programming Assignment 1 – Recuperação de Informação. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Recuperação de Informação - PA1)*. ACM, New York, NY, USA, 2 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introdução

O primeiro trabalho prático proposto na disciplina de recuperação de informação implementa um Crawler, cujo objetivo é coletar informações de diferentes páginas web, a partir da entrada do usuário. Desse forma, o sistema precisa além de coletar as páginas, ser robusto e eficiente para lidar com diferentes situações durante a coleta. As seções a seguir descrevem mais sobre a implementação, estratégias utilizadas e resultados obtidos do trabalho.

2 Metodologia

A abordagem utilizada para construção do crawler foi feita seguindo a estrutura abaixo:

- Etapa1: Adição das URLs do usuário na fila de fronteira
- Etapa2: Pre processamento da URL e verificações
- Etapa3: Coleta do conteúdo e processamento
- Etapa4: Coleta de novos links e armazenamento da página

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Recuperação de Informação - PA1, Abril 28, 2025, Belo Horizonte, MG

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/XXXXXXX.XXXXXXX>

Comentando sobre a estrutura geral do sistema, sobre a etapa 1, enquanto a fila não estiver vazia ou não foi atingido o limite de coleta, o processo continua. Desse modo, o link inicial é obtido e são feitas filtragem e verificações (se o link é válido e existe, além da normalização da URL). É verificado também se o link já foi visitado anteriormente (evitando assim URLs repetidas e ciclos de visitação).

Após realizar as etapas 1 e 2, o conteúdo então é obtido via requisição a URL, sendo tomado o cuidado de lidar com erros que podem ocorrer durante o processo. O conteúdo obtido então é convertido para HTML e conteúdos inválidos são descartados. Vale destacar que essa etapa só ocorre após a coleta do arquivo 'robots.txt', que determina se a URL pode ser coletada, além da espera pelo delay estipulado (se nenhum delay foi delimitado, utiliza-se o tempo de 0.1 segundos para novos domínios).

Após a coleta e processamento da etapa 3, inicia-se a etapa 4, em que é feita a coleta dos links de saída presentes na página coletada. Desse modo, os links obtidos são divididos em três filas, de links brasileiros, internos e externos, para então dar-se continuidade ao processo enquanto as condições iniciais forem satisfeitas. Por fim, o corpus é salvo no formato WARC.

Ademais, o código foi paralelizado utilizando threads. Isso foi feito para aumentar a velocidade de todo o processo, com o número de threads mantido em 30. As estratégias descritas acima foram obtidas através de experimentação e testes do código, sendo comentadas mais abaixo.

Vale destacar por fim que a complexidade final (aproximada) do código desenvolvido é:

$$O(U * (L + C))$$

em que:

U = quantidade de links na fronteira (representa cada URL que possivelmente será coletada)

L = quantidade de links presentes na página atual que está sendo coletada

C = custo das operações de verificações, processamento e coleta do conteúdo das páginas

3 Testes realizados e resultados

Os testes foram realizados em uma máquina com Debian 12, kernel 6.1.0, CPU Intel i5 - 11ª geração e 16 GB de ram. A coleta de 100.000 páginas demorou cerca de 22 horas, totalizando 4.9 GiB. A seed inicial foi definida como sendo a união de todas as seeds passadas no enunciado do trabalho.

Comentando sobre a decisão de utilizar três filas (uma para links brasileiros, internos e outra para externos), sem a separação entre links externos e internos, o crawler obteve resultados ruins não descobrindo as 100.000 páginas ou ficando "preso" em domínios com muitas subpáginas. Já sobre a fila de links brasileiros, isso foi válido

pois eventualmente eram descobertos links de outros continentes, o que piorava o tempo de execução. Desse modo, o crawler final utiliza 60% das threads para links brasileiros e o restante dividido entre links internos e externos, sendo este o balanço encontrado entre descoberta x exploração x performance.

Já sobre o número de threads, o valor também foi testado de modo a conseguir uma boa performance, além de não consumir todos os recursos da máquina utilizada para desenvolvimento do trabalho. A tabela 1 mostra os resultados que levaram a essa escolha. Os testes foram feitos considerando o tempo para baixar 100 páginas, com as medições sendo feitas a cada conjunto de 10 URLs coletadas. Todos os valores de tempo são médias, sendo válido comentar que alguns blocos foram desconsiderados por demorarem muito mais que os demais (por conta de erros de requisições). Dessa forma, o número de threads considerado no trabalho foi de 30, pois valores maiores acabaram levando a piores resultados de performance.

# Threads	Tempo 10 URLs (s)	Tempo total (s)
1	19	102
2	10	91
3	8	49
10	3	39
20	2	23
30	1	28
40	0.8	65

Table 1: Desempenho do crawler com diferentes números de threads

A figura 1 apresentada abaixo mostra a porcentagem de páginas coletadas por cada quantidade de tokens. O número de tokens foi calculado tendo como base o número de palavras no corpo do texto (removendo caracteres de código e blocos HTML). Percebe-se que o corpus coletado possui uma grande quantidade de páginas com um elevado número de tokens (mais de 50% do corpus coletado possui mais de 5.000 tokens). Vale destacar também que a contagem de tokens das páginas é bem variável no geral. Tal ideia é comentada levando em consideração o fato de muitas páginas possuírem conteúdos dinâmicos, assim, a coleta da URL retorna apenas elementos de código Javascript, ao invés do conteúdo HTML. Como os resultados são muito grandes, não serão mostrados exemplos disso nesta documentação, porém é possível de ser conferido no corpus coletado.

Já sobre o número de páginas por domínio, a distribuição está mencionada na tabela 2. É possível perceber que os sites "kids britannica", "purepeople" e "pinterest" foram os que mais apresentaram páginas internas. Esse resultado é válido na prática, pois ao conferir esses domínios, percebe-se que eles tratam, respectivamente, de: conteúdos educacionais (muitos links para atividades e imagens); conteúdos de fofoca e notícias de celebridade; além de um portal de imagens e trabalhos visuais, justificando assim o grande número de conteúdos vinculados. Vale destacar também que o top 10 representa menos de 1% do corpus coletado, mostrando como a imensa maioria dos domínios possui menor quantidade de ramificações.

4 Execução

Para executar o programa, basta seguir:

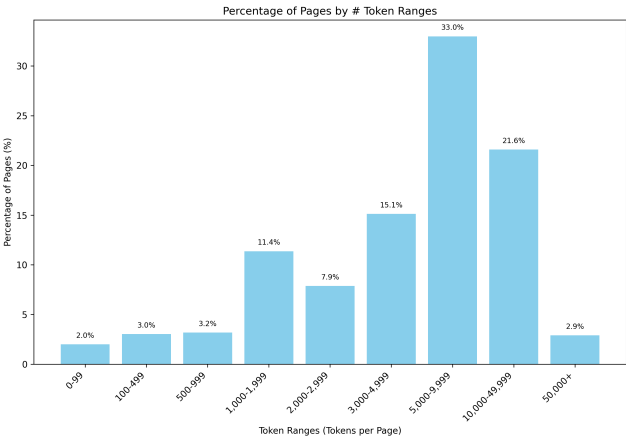


Figure 1: Porcentagem de páginas por número de tokens

Domínio	Número de subpáginas
kids.britannica.com	5269
www.purepeople.com.br	3435
pinterest.com	1690
www.architonic.com	1562
www.mpf.mp.br	1143
www.brookings.edu	1069
pursuit.unimelb.edu.au	857
starbucks.com.br	827
www.britannica.com	820
support.google.com	676

Table 2: Top 10 domínios com maior número de subpáginas

```
python3 -m venv <nome do ambiente>
source <caminho até o ambiente>/bin/activate
pip install -r requirements
python3 main.py -s <seeds> -n <limite> [-d]
```

Caso necessário imprimir mais informações de execução, as flags adicionais "-c" e "-sr" podem ser acrescentadas, habilitando, respectivamente, a impressão de logs de execução do código e o salvamento em arquivos .json das estatísticas de execução do crawler.

5 Conclusão

Percebe-se agora com a execução da tarefa como o trabalho ampliou o entendimento sobre crawlers e máquinas de busca. Os principais desafios foram a grande quantidade de erros ao coletar os conteúdos, além do cuidado para eficiência do código, sendo estes desafios superados após diversas execuções. Desse modo, o objetivo de coletar as 100.000 páginas foi cumprido considerando todas as restrições apresentadas.